

Université Mohammed Premier
École Nationale des Sciences Appliquées d'Oujda
Filière Génie Informatique – GI3

Rapport de Mini-Projet

[uniSchool]

Réalisé par : Chaimae Alla
Nour Moulid

Encadrante : Mme Douae EL HILA

Module : Développement Java IHM
Année : 2025/2026

Remerciements

Au terme de ce projet, nous tenons à adresser nos plus sincères remerciements à toutes les personnes ayant contribué, de près ou de loin, à sa réalisation.

Nous exprimons tout particulièrement notre profonde gratitude à notre encadrante, **Madame Douae EL HILA**, pour son accompagnement constant, sa disponibilité, ainsi que la pertinence de ses conseils et de ses orientations. Son encadrement rigoureux et son soutien tout au long de ce projet nous ont permis d'approfondir nos connaissances et de développer nos compétences dans le domaine du développement Java et JavaFX.

Nous adressons également nos remerciements à l'ensemble des enseignants du module **Développement Java IHM** pour la qualité de leur enseignement, leur engagement pédagogique et les connaissances qu'ils nous ont transmises. Les compétences acquises grâce à leur formation ont constitué une base essentielle pour la réalisation de ce travail.

Enfin, nous remercions l'établissement et le corps pédagogique pour avoir mis à notre disposition un environnement d'apprentissage propice au développement de nos compétences académiques et techniques.

À toutes ces personnes, nous adressons l'expression de notre profonde reconnaissance et de notre respect.

Table des matières

Remerciements	1
Introduction	6
1 Présentation du projet	7
1.1 Description de l'application.....	7
1.2 Besoins fonctionnels.....	7
1.3 Besoins non fonctionnels	7
2 Conception	8
2.1 Architecture MVC.....	8
2.2 Diagramme de classes UML	8
2.3 Diagramme de cas d'utilisation.....	8
2.4 Diagramme de séquence (optionnel).....	8
3 Environnement de travail	9
3.1 Outils et technologies utilisés	9
3.2 Structure du projet	9
4 Répartition des tâches	10
4.1 Tâches de l'étudiant 1 – [Prénom Nom]	10
4.2 Tâches de l'étudiant 2 – [Prénom Nom]	10
5 Explication du code	11
5.1 Connexion à la base de données – Database.java.....	11
5.2 Les classes modèles.....	11
5.3 Les classes DAO	11
5.4 Les contrôleurs.....	11
5.5 Les fichiers FXML	11
5.6 Fonctionnalité spécifique au projet.....	11
6 Interfaces de l'application	12
6.1 Fenêtre principale.....	12
6.2 Interfaces de gestion de l'entité 1	12
6.3 Interfaces de gestion de l'entité 2	12
6.4 Tableau de bord – Statistiques.....	12
6.5 Menus et dialogues	12
6.6 Export de données.....	12
Conclusion	13

Table des figures

2.1.1 Architecture MVC	8
2.2.1 Diagramme de classes	8
2.3.1 Diagramme de cas d'utilisation	8
3.2.1 Architecture détaillé du projet	9

Liste des tableaux

4.1	Tâches réalisées par Alla Chaimae	10
4.2	Tâches réalisées par Moulid Nour	10
1.2.1	Répartition des fonctionnalités par rôle	7

Introduction

Dans le cadre du module **Développement Java IHM**, nous avons été amenés à concevoir et à réaliser une application de gestion scolaire baptisée **UniSchool**. Ce projet a pour objectif de mettre en pratique les connaissances acquises en programmation Java, en conception d'interfaces graphiques et en gestion de bases de données, à travers le développement d'une application complète et fonctionnelle.

Le système développé vise à faciliter la gestion des activités académiques d'un établissement d'enseignement supérieur. Il repose sur une interface graphique réalisée avec **JavaFX** et une base de données **MySQL**, assurant ainsi une gestion efficace et sécurisée des informations. Les principales entités prises en compte sont les **étudiants**, les **enseignants**, les **administrateurs** et les **notes**, permettant une administration centralisée des données académiques.

L'application offre des fonctionnalités adaptées à chaque catégorie d'utilisateurs. Les administrateurs disposent d'outils de gestion et de supervision, les enseignants peuvent consulter et gérer les informations pédagogiques, tandis que les étudiants ont accès à leurs données académiques et à leurs résultats.

Ce rapport présente les différentes étapes de réalisation du projet, depuis l'analyse des besoins et la conception du système jusqu'à son implémentation et sa mise en œuvre. Il décrit également les choix techniques adoptés, l'architecture de l'application, les fonctionnalités développées ainsi que les interfaces réalisées.

Réalisé en binôme, ce projet nous a permis de renforcer nos compétences techniques, de nous familiariser avec les bonnes pratiques du développement logiciel et d'acquérir une expérience concrète dans la conception d'applications de gestion orientées utilisateur.

Chapitre 1

Présentation du projet

1.1 Description de l'application

1.2 Besoins fonctionnels

1.3 Besoins non fonctionnels

Chapitre 1

1.1 Description de l'application

UniSchool est une application de gestion scolaire destinée aux établissements d'enseignement supérieur. Elle permet de gérer de manière centralisée et efficace les étudiants, les enseignants, les matières, les notes, les absences, les messages et les emplois du temps.

L'application propose trois interfaces adaptées aux besoins spécifiques de chaque rôle :

- **Administrateur** : Dispose d'une vue globale et d'un contrôle total sur l'ensemble des données. Il peut gérer les étudiants, les enseignants, les matières, les notes, les absences, les emplois du temps et la communication interne.
- **Enseignant** : Peut consulter ses cours, gérer les notes de ses étudiants, visualiser les statistiques de ses classes et communiquer avec l'administration ainsi qu'avec les étudiants.
- **Étudiant** : Peut consulter son bulletin de notes, ses moyennes, ses absences et son classement. Il peut également communiquer avec l'administration et ses enseignants.

Grâce à cette architecture orientée rôles, **UniSchool** offre une expérience utilisateur personnalisée et sécurisée, adaptée aux besoins de chaque acteur de la communauté universitaire.

Chapitre 1

1.2 Besoins fonctionnels

L'application **UniSchool** offre un ensemble complet de fonctionnalités destinées à simplifier la gestion scolaire et à améliorer l'organisation des activités académiques.

1. Authentification et gestion des rôles

L'application intègre un système d'authentification sécurisée permettant à chaque utilisateur (administrateur, enseignant ou étudiant) d'accéder à un espace adapté à son rôle, garantissant ainsi la confidentialité et la sécurité des données.

2. Gestion des entités principales

UniSchool permet la gestion complète des étudiants et des enseignants à travers les opérations d'ajout, de consultation, de modification et de suppression des informations.

3. Gestion des notes et des absences

L'application assure la gestion des notes avec le calcul automatique des moyennes, ainsi que le suivi des absences des étudiants, permettant un contrôle précis de leur assiduité et de leurs performances académiques.

4. Gestion des emplois du temps

Un module dédié à la gestion des emplois du temps facilite l'organisation des cours selon les filières et les niveaux d'études, offrant une vision claire et structurée des activités pédagogiques.

5. Recherche et filtrage

Des outils de recherche et de filtrage avancés permettent d'accéder rapidement aux informations souhaitées, améliorant ainsi l'efficacité de la gestion quotidienne.

6. Communication intégrée

Un espace de communication favorise les échanges entre les différents utilisateurs de la plateforme, renforçant la collaboration au sein de la

communauté universitaire.

7. Statistiques et exportation

Enfin, l'application met à disposition un tableau de bord statistique

Chapitre 1

permettant de visualiser différents indicateurs académiques, ainsi qu'une fonctionnalité d'exportation des données au format CSV pour faciliter leur exploitation et leur archivage.

Fonctionnalité	Administrateur	Enseignant	Étudiant
Gestion des étudiants	oui	non	non
Gestion des enseignants	oui	oui	non
Gestion des notes	oui	oui	non
Consultation des notes	oui	oui	oui
Gestion des absences	oui	non	non
Consultation des absences	oui	oui	oui
Gestion de l'emploi du temps	oui	non	non
Consultation de l'emploi du temps	oui	oui	non
Communication	oui	oui	oui
Statistiques	oui	oui	non
Export CSV	oui	oui	non

1.2.1 Répartition des fonctionnalités par rôle

1.3 Besoins non fonctionnels

Afin d'assurer la qualité et l'efficacité de l'application **UniSchool**, plusieurs besoins non fonctionnels ont été pris en compte lors de sa conception. Tout d'abord, l'application doit garantir de bonnes **performances** en assurant une

Chapitre 1

exécution rapide des différentes opérations et une réponse fluide aux actions des utilisateurs.

La **sécurité** constitue également un aspect essentiel du système. Grâce à un mécanisme d'authentification, chaque utilisateur accède uniquement aux fonctionnalités correspondant à son rôle et à ses autorisations.

L'**ergonomie** de l'application a été particulièrement soignée afin d'offrir une interface graphique intuitive, moderne et facile à utiliser, développée avec les composants **JavaFX**. De plus, la **persistance des données** est assurée par une base de données MySQL permettant un stockage fiable et sécurisé des informations.

Enfin, l'application a été conçue selon une architecture **MVC (Modèle-Vue-Contrôleur)** garantissant une bonne **maintenabilité**, une meilleure organisation du code et une évolution plus facile du système.

Chapitre 2

Conception

2.1 Architecture MVC

2.2 Diagramme de classes UML

2.3 Diagramme de cas d'utilisation

Chapitre 2

2.1 Architecture MVC

L'application **UniSchool** est développée selon l'architecture **MVC (Modèle-Vue-Contrôleur)**, un modèle de conception qui permet de séparer les différentes responsabilités du système afin de faciliter son organisation, sa maintenance et son évolution.

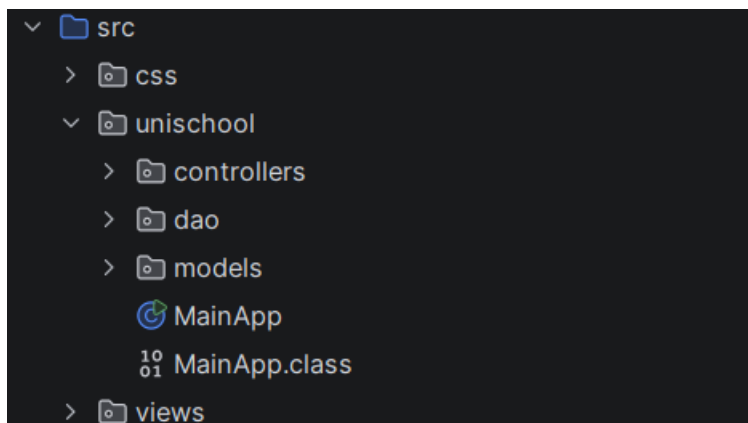
La couche **Modèle** regroupe les classes représentant les entités métier de l'application, telles que les étudiants, les enseignants, les notes ou encore les absences. Ces classes contiennent les données manipulées par le système.

La couche **Vue** correspond à l'ensemble des interfaces graphiques développées avec **JavaFX** et définies dans des fichiers **FXML**. Elle assure l'interaction visuelle avec l'utilisateur et l'affichage des informations.

La couche **Contrôleur** joue le rôle d'intermédiaire entre la vue et le modèle. Elle contient la logique de traitement des actions effectuées par l'utilisateur et coordonne les échanges entre les différentes couches de l'application.

Enfin, la couche **DAO (Data Access Object)** est responsable de l'accès à la base de données MySQL. Elle permet d'effectuer les opérations de consultation, d'insertion, de modification et de suppression des données tout en isolant la logique d'accès aux données du reste de l'application.

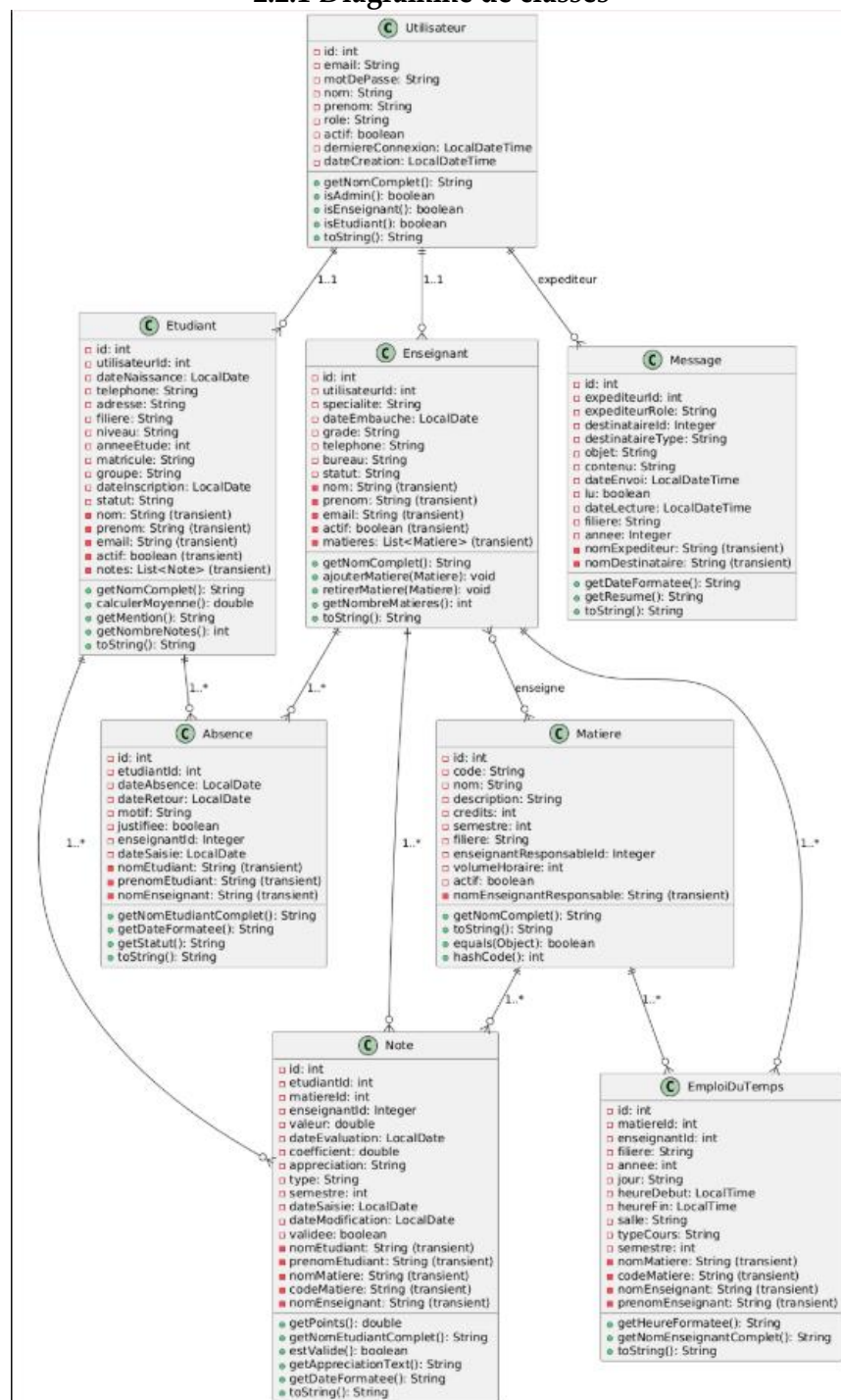
Cette architecture favorise une meilleure modularité du projet, améliore sa maintenabilité et facilite l'ajout de nouvelles fonctionnalités.



Chapitre 2

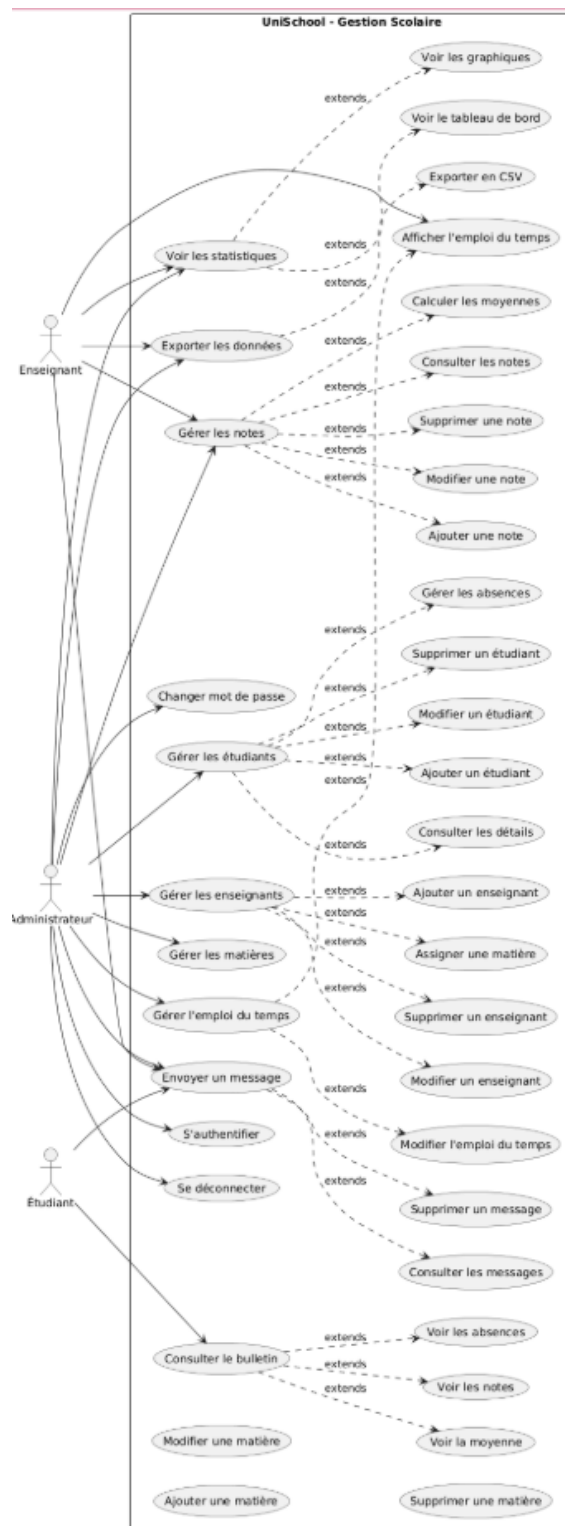
2.2 Diagramme de classes UML

2.2.1 Diagramme de classes



2.3 Diagramme de cas d'utilisation

Chapitre 2



2.3.1 Diagramme de cas d'utilisation

Chapitre 3

Environnement de travail

3.1 Outils et technologies utilisés

3.2 Structure du projet

Chapitre 3

3.1 Outils et technologies utilisés

Le développement de l'application **UniSchool** a nécessité l'utilisation de plusieurs outils et technologies. Le langage **Java 21 LTS** a été choisi pour l'implémentation de l'application, tandis que **JavaFX 21.0.1** a été utilisé pour la conception des interfaces graphiques. La gestion des données repose sur une base de données **MySQL 8.0.33**, connectée à l'application grâce au pilote **MySQL Connector/J 8.0.33**.

L'environnement de développement utilisé est **IntelliJ IDEA 2026.1.3**, offrant des outils avancés pour la programmation et le débogage. L'administration de la base de données a été réalisée à l'aide de **phpMyAdmin 5.2.1**, exécuté sur un serveur local **XAMPP 3.3.0**. Enfin, le suivi des versions et la collaboration ont été assurés grâce à **Git** et **GitHub**.

Chapitre 3

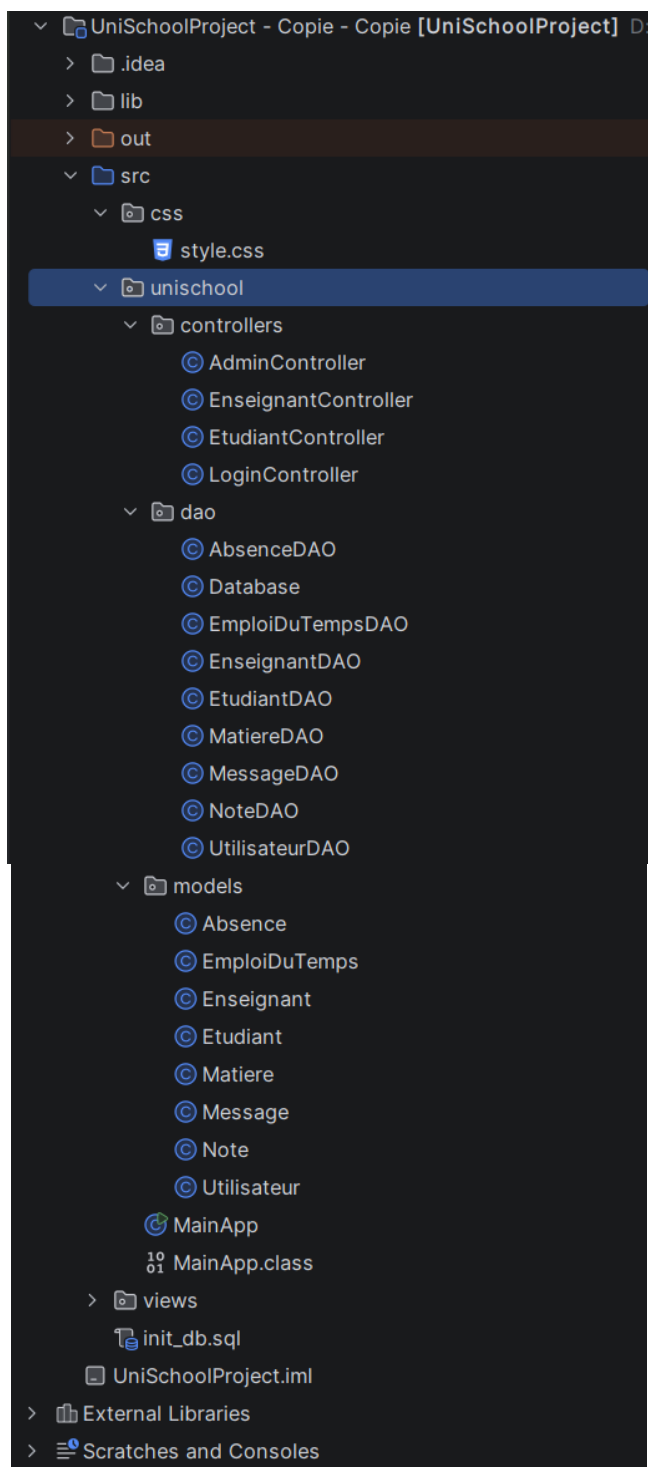
3.2 Structure du projet

Le projet **UniSchool** est organisé en plusieurs packages afin de garantir une bonne modularité et une meilleure maintenabilité du code. Le package **models** regroupe les différentes entités métier de l'application, telles que les utilisateurs, les étudiants, les enseignants, les matières, les notes, les absences, les messages et les emplois du temps.

Le package **dao** contient les classes responsables de l'accès à la base de données MySQL et de la gestion des opérations CRUD. Les traitements liés aux interactions utilisateur sont assurés par les classes du package **controllers**, tandis que les interfaces graphiques développées avec JavaFX sont regroupées dans le package **views** sous forme de fichiers FXML.

Les feuilles de style sont centralisées dans le dossier **css**, permettant d'assurer une apparence homogène de l'application. La classe **MainApp** constitue le point d'entrée du programme. Enfin, le projet comprend également les bibliothèques externes nécessaires, le script d'initialisation de la base de données, la documentation du projet ainsi que le fichier de configuration Maven.

Chapitre 3



3.2.1 Architecture détaillé du projet

Chapitre 4

Répartition des tâches

4.1 Tâches de l'étudiant 1 – [Alla Chaimae]

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception de la base de données	init_db.sql	terminé
Modèles (Models)	models/*.java	
DAO (Accès aux données)	dao/*.java	
Interface Administrateur	main_admin.fxml AdminController.java	
Interface Enseignant	main_enseignant.fxml EnseignantController.java	
Gestion des statistiques	AdminController.java	
Export CSV	EnseignantController.java AdminController.java	
Communication (messages)	Message.java MessageDAO.java	
Emploi du temps	EmploiDuTemps.java EmploiDuTempsDAO.java	

TABLE 4.1 – Tâches réalisées par Alla chaimae

4.2 Tâches de l'étudiant 2 – [Moulid Nour]

Chapitre 4

Tâche réalisée	Fichier(s) concerné(s)	Statut
Interface de connexion	login.fxml	terminé
Interface Étudiant	LoginController.java	
	main_etudiant.fxml	
Feuille de style CSS	EtudiantController.java	
	style.css	
Gestion des absences	EtudiantController.java	
Calcul des moyennes et mentions	Etudiant.java	
	EtudiantController.java	
Rédaction du rapport	rapport.pdf	

TABLE 4.2 – Tâches réalisées par Moulid Nour

Chapitre 5

Explication du code

5.1 Connexion à la base de données – Database.java

5.2 Les classes modèles

5.3 Les classes DAO

5.4 Les contrôleurs

5.5 Les fichiers FXML

5.6 Fonctionnalité spécifique au projet

Chapitre 5

5.1 Connexion à la base de données – Database.java

La classe **Database.java** gère la connexion à la base de données **MySQL**. Elle utilise le pattern **Singleton** pour garantir qu'une seule et unique connexion est établie tout au long de l'exécution de l'application, évitant ainsi la multiplication des connexions et optimisant les performances.

5.1.1 Rôle de la classe

Cette classe centralise les paramètres de connexion à la base de données, à savoir l'URL de connexion, le nom d'utilisateur et le mot de passe. Elle fournit également une méthode statique permettant d'obtenir une connexion active, en la créant si elle n'existe pas ou si elle a été fermée.

5.1.2 Pattern Singleton

Le pattern Singleton est implémenté à travers une variable statique `connection` qui conserve l'unique instance de la connexion. La méthode `getConnection()` vérifie si la connexion est null ou fermée avant d'en établir une nouvelle, garantissant ainsi qu'une seule connexion est utilisée pour toutes les opérations sur la base de données.

5.1.3 Gestion des erreurs

La classe intègre une gestion des erreurs permettant de capturer les exceptions liées à l'absence du pilote JDBC ou à des problèmes de connexion. Un message explicite est affiché en cas d'échec, facilitant ainsi le débogage.

5.1.4 Méthodes principales

- **getConnection()** : Retourne la connexion active à la base de données. Si la connexion est null ou fermée, une nouvelle est créée.
- **closeConnection()** : Ferme la connexion de manière explicite, libérant ainsi les ressources.
- **testConnection()** : Permet de vérifier si la connexion à la base de données est fonctionnelle.

5.1.5 Utilisation dans les DAO

Les classes DAO utilisent la méthode `getConnection()` pour obtenir la connexion à la base de données et effectuer les opérations CRUD. Cette approche assure une gestion centralisée et cohérente de la connexion à travers toute l'application.

5.2 Les classes modèles

Chapitre 5

Les classes modèles (ou entités) représentent les différentes données manipulées par l'application. Chaque classe correspond à une table de la base de données et encapsule les attributs ainsi que les méthodes de calcul et de validation associées.

Les principales classes modèles sont :

Utilisateur.java : Représente un utilisateur du système (administrateur, enseignant ou étudiant). Cette classe contient les informations communes telles que l'identifiant, l'email, le mot de passe, le nom, le prénom et le rôle. Elle fournit également des méthodes pour vérifier le rôle de l'utilisateur.

Etudiant.java : Hérite des informations de l'utilisateur et ajoute des attributs spécifiques tels que la date de naissance, le téléphone, la filière, le niveau, l'année d'étude, le matricule et le statut. Cette classe implémente des méthodes de calcul comme calculerMoyenne() et getMention().

Enseignant.java : Représente un enseignant avec des attributs spécifiques comme la spécialité, la date d'embauche, le grade et le statut. Elle est associée à une liste de matières enseignées.

Matiere.java : Définit une matière avec son code, son nom, sa description, son nombre de crédits, son semestre et sa filière associée. Elle peut également référencer un enseignant responsable.

Note.java : Représente la note obtenue par un étudiant dans une matière donnée. Elle contient la valeur de la note, le coefficient, la date d'évaluation, le type d'évaluation et une éventuelle appréciation.

Absence.java : Enregistre les absences des étudiants avec la date de l'absence, le motif et l'enseignant ayant constaté l'absence.

Message.java : Permet la gestion des messages échangés entre les utilisateurs, avec des informations comme l'expéditeur, le destinataire, l'objet, le contenu et la date d'envoi.

EmploiDuTemps.java : Représente l'emploi du temps d'une filière avec les informations relatives aux cours (matière, enseignant, jour, horaire, salle, type de cours).

Chapitre 5

5.3 Les classes DAO

Les classes DAO (Data Access Object) assurent l'interface entre l'application et la base de données. Elles encapsulent toutes les opérations d'accès aux données, garantissant une séparation claire entre la logique métier et la persistance.

Chaque classe DAO correspond à une entité et implémente les opérations CRUD (Create, Read, Update, Delete). Les principales classes DAO sont :

Database.java : Gère la connexion à la base de données MySQL en utilisant le pattern Singleton.

UtilisateurDAO.java : Gère les opérations liées aux utilisateurs : authentification, création, modification, suppression et recherche.

EtudiantDAO.java : Gère les opérations CRUD pour les étudiants, ainsi que des méthodes de recherche par filière, par année ou par matricule.

EnseignantDAO.java : Gère les opérations CRUD pour les enseignants et permet d'associer ou de dissocier des matières.

MatiereDAO.java : Gère les opérations CRUD pour les matières, avec des méthodes de recherche par semestre ou par filière.

NoteDAO.java : Gère les opérations CRUD pour les notes et fournit des méthodes de calcul des moyennes (moyenne générale, moyenne par étudiant, moyenne par matière).

AbsenceDAO.java : Gère les opérations CRUD pour les absences, avec des méthodes de comptage par étudiant.

MessageDAO.java : Gère les opérations CRUD pour les messages, avec des méthodes de filtrage par destinataire et par expéditeur.

EmploiDuTempsDAO.java : Gère les opérations CRUD pour les emplois du temps, avec des méthodes de recherche par filière, par année ou par enseignant.

Chapitre 5

5.4 Les contrôleurs

Les contrôleurs constituent la couche intermédiaire entre l'interface utilisateur (vue) et les données (modèle). Ils sont responsables de la gestion des événements utilisateur, du traitement des actions et de la mise à jour de l'interface.

Les principaux contrôleurs sont :

LoginController.java : Gère l'authentification des utilisateurs. Il valide les identifiants saisis, vérifie les droits d'accès selon le rôle et redirige l'utilisateur vers l'interface appropriée (administrateur, enseignant ou étudiant).

AdminController.java : Gère l'ensemble des fonctionnalités réservées à l'administrateur : gestion des étudiants, des enseignants, des notes, des absences, des emplois du temps et des messages. Il intègre également les statistiques et l'export des données.

EnseignantController.java : Gère les fonctionnalités accessibles à l'enseignant : consultation de l'emploi du temps, gestion des notes de ses cours, visualisation des statistiques de ses classes et communication.

EtudiantController.java : Gère les fonctionnalités accessibles à l'étudiant : consultation du bulletin de notes, de la moyenne, des absences et des messages reçus.

5.5 Les fichiers FXML

Les fichiers **FXML** décrivent la structure des interfaces graphiques de l'application. Ils sont créés à l'aide de **Scene Builder** et permettent de séparer la présentation de la logique métier. Ces fichiers sont chargés par les contrôleurs correspondants.

Chapitre 5

Les principaux fichiers FXML sont :

- **login.fxml** : Interface de connexion avec les champs email, mot de passe et le bouton d'authentification.
- **main_admin.fxml** : Interface principale de l'administrateur, structurée en onglets : Gestion Académique, Enseignants, Emploi du temps, Notes, Communication et Statistiques.
- **main_enseignant.fxml** : Interface principale de l'enseignant, structurée en onglets : Emploi du temps, Gestion des Notes, Statistiques et Communication.
- **main_etudiant.fxml** : Interface principale de l'étudiant, structurée en onglets : Mon Bulletin et Communication.

5.6 Fonctionnalité spécifique au projet

1. Gestion des moyennes et des mentions

Chaque étudiant bénéficie d'un calcul automatique de sa moyenne générale à partir de ses notes. La mention correspondante (Très Bien, Bien, Assez Bien, Passable ou Insuffisant) est également déterminée selon le barème en vigueur.

Chapitre 5

2. Tableau de bord statistique

L'interface administrateur propose un tableau de bord avec des indicateurs visuels : nombre d'étudiants, d'enseignants, de matières, de notes, moyenne générale, taux de réussite, total des absences et meilleur étudiant. Des graphiques illustrent la répartition des étudiants par filière et la distribution des notes.

3. Gestion des absences

Les enseignants peuvent enregistrer les absences des étudiants avec la possibilité de les justifier ou non. L'administrateur peut consulter et gérer l'ensemble des absences.

4. Communication intégrée

Un système de messagerie permet aux différents utilisateurs d'échanger des messages de manière interne. L'administrateur peut envoyer des messages à tous les étudiants, à tous les enseignants ou à des groupes spécifiques. Les enseignants et les étudiants peuvent également communiquer entre eux.

5. Export des données

L'application permet d'exporter les données au format CSV, facilitant ainsi leur exploitation externe, leur analyse et leur archivage.

6. Emploi du temps

Les enseignants peuvent consulter leur emploi du temps par semestre, tandis que l'administrateur peut gérer les emplois du temps par filière et par année.

7. Recherche et filtrage

Des outils de recherche et de filtrage sont disponibles dans les différents onglets pour faciliter l'accès aux informations souhaitées (recherche par nom, filtre par filière, par année, par matière, etc.).

Chapitre 6

Interfaces de l'application

6.1 Fenêtre principale

6.2 Interfaces de gestion de l'entité 1

6.3 Interfaces de gestion de l'entité 2

6.4 Tableau de bord – Statistiques

6.5 Menus et dialogues

6.6 Export de données

Chapitre 6

6.1 Fenêtre principale

L'application **UniSchool** s'ouvre sur une **fenêtre de connexion** qui constitue le point d'entrée unique du système. Cette interface épurée permet à chaque utilisateur de s'authentifier en saisissant son adresse email et son mot de passe. Elle affiche également les comptes de test pour faciliter la démonstration.

Une fois authentifié, l'utilisateur est redirigé vers une interface adaptée à son rôle :

- **Administrateur** : Accès à l'ensemble des fonctionnalités de gestion
- **Enseignant** : Accès à ses cours, ses notes et ses statistiques
- **Étudiant** : Accès à son bulletin et à ses messages

Chaque interface principale est structurée autour d'une **barre de titre** affichant le nom de l'utilisateur connecté, des boutons de rafraîchissement et de déconnexion, ainsi qu'une **barre de statut** en bas de fenêtre indiquant l'état de l'application.

Chapitre 6

6.2 Interfaces de gestion de l'entité 1 – Gestion des étudiants

L'interface de gestion des étudiants est accessible depuis l'onglet "**Gestion Académique**" de l'interface administrateur. Elle offre un ensemble complet de fonctionnalités :

Filtrage et recherche :

- Filtrage par filière et par année d'étude
- Recherche rapide par nom, prénom ou email
- Affichage du nombre total d'étudiants filtrés

Gestion des données :

- Tableau récapitulatif affichant l'ID, le nom, le prénom, l'email, la filière, l'année, le nombre d'absences, la moyenne et le statut de chaque étudiant
- Boutons d'action pour **Ajouter**, **Modifier** et **Supprimer** un étudiant
- Possibilité de gérer les absences d'un étudiant sélectionné

Détails de l'étudiant :

- Un panneau déroulant affiche les informations détaillées de l'étudiant sélectionné : nom, filière, moyenne, mention, nombre d'absences
- Un tableau présente le relevé de notes détaillé de l'étudiant avec les matières, notes, coefficients, points, dates et appréciations

6.3 Interfaces de gestion de l'entité 2 – Gestion des notes

L'interface de gestion des notes est accessible depuis l'onglet "**Gestion des Notes**"

Chapitre 6

aussi bien pour l'administrateur que pour l'enseignant.

Fonctionnalités communes :

- Tableau complet des notes avec les colonnes : ID, Étudiant, Matière, Note, Coefficient, Points, Type, Date, Appréciation et Validée
- Filtrage par matière et par étudiant
- Export des données au format CSV

Fonctionnalités spécifiques à l'administrateur :

- Vue globale de toutes les notes de l'établissement
- Affichage de la moyenne générale
- Ajout, modification et suppression de n'importe quelle note

Fonctionnalités spécifiques à l'enseignant :

- Vue restreinte aux notes de ses propres cours
- Ajout, modification et suppression des notes de ses étudiants
- Validation des notes saisies

6.4 Tableau de bord - Statistiques

Le tableau de bord statistique offre une vue synthétique des indicateurs clés de l'établissement. Il est accessible depuis l'onglet "**Statistiques**" de l'interface administrateur.

Chapitre 6

Indicateurs chiffrés :

- Nombre total d'étudiants
- Nombre total d'enseignants
- Nombre total de matières
- Nombre total de notes
- Moyenne générale
- Taux de réussite (pourcentage de notes supérieures ou égales à 10/20)
- Total des absences
- Meilleur étudiant (avec sa moyenne)

Visualisations graphiques :

- **Diagramme à barres** : Répartition des notes par matière, permettant d'identifier les matières où les étudiants excellent ou rencontrent des difficultés
- **Camembert** : Répartition des étudiants par filière, offrant une vision de la distribution des effectifs
- **Moyenne par matière** : Visualisation des performances moyennes dans chaque matière

Listes détaillées :

- Répartition des étudiants par filière et par année
- Répartition des notes par catégorie (Excellent, Bien, Assez Bien, Passable, Insuffisant)

6.5 Menu et dialogue

Chapitre 6

Menus :

L'application intègre une barre de menus dans l'interface administrateur :

- **Fichier** : Export des données, Quitter
- **Gestion** : Accès rapide aux sections Étudiants, Enseignants, Notes
- **Statistiques** : Accès au tableau de bord
- **Aide** : Informations sur l'application

Dialogues :

L'application utilise différents types de dialogues pour interagir avec l'utilisateur :

- **Dialogues de confirmation** : Apparaissent avant toute suppression d'une donnée (étudiant, enseignant, note, absence, message) pour éviter les suppressions accidentelles
- **Dialogues d'erreur** : Alertes en cas de saisie incorrecte ou d'erreur système
- **Dialogues de succès** : Notifications après une opération réussie (ajout, modification, suppression)
- **Dialogues de formulaire** : Fenêtres modales pour l'ajout ou la modification d'une entité (étudiant, enseignant, note, absence)

Tooltips :

Des info-bulles sont présentes sur les champs de saisie et les boutons pour guider l'utilisateur dans l'utilisation de l'application.

6.6 Export de données

L'application propose une fonctionnalité d'export des données au format CSV, accessible depuis plusieurs interfaces :

Export des notes :

- Accessible depuis l'onglet "Gestion des Notes"

Chapitre 6

- Exporte l'ensemble des notes affichées dans le tableau
- Colonnes exportées : Étudiant, Matière, Note, Coefficient, Points, Type, Date, Appréciation, Validée
- Fichier nommé automatiquement avec la date et l'heure

Export de l'emploi du temps :

- Accessible depuis l'onglet "Emploi du temps"
- Exporte les cours affichés pour la filière et l'année sélectionnées

Export du bulletin (optionnel) :

- Exporte le bulletin d'un étudiant spécifique

Fonctionnement :

Un FileChooser est utilisé pour permettre à l'utilisateur de sélectionner l'emplacement et le nom du fichier à exporter. Les données sont écrites dans un fichier CSV avec un séparateur point-virgule (;), garantissant une compatibilité avec les logiciels de tableur comme Microsoft Excel et LibreOffice Calc.

Conclusion

Au terme de ce projet, nous avons conçu et développé UniSchool, une application de gestion scolaire permettant de centraliser et de faciliter la gestion des étudiants, des enseignants, des notes, des absences et des emplois du temps. Ce projet nous a permis de mettre en pratique les connaissances acquises dans le cadre du module Développement Java IHM, notamment en programmation Java, en conception d'interfaces graphiques avec JavaFX et en gestion de bases de données MySQL.

La réalisation de cette application nous a également permis d'appliquer l'architecture MVC, garantissant une séparation claire entre les différentes couches du système. Grâce à ses différentes fonctionnalités, telles que la gestion des utilisateurs selon leurs rôles, les opérations CRUD, les statistiques graphiques et l'exportation des données, l'application répond aux principaux besoins d'un établissement d'enseignement supérieur.

Au cours du développement, nous avons été confrontés à plusieurs défis techniques, notamment la configuration de l'environnement JavaFX, la gestion des connexions à la base de données, l'implémentation des graphiques statistiques ainsi que la synchronisation des données entre l'interface utilisateur et la base de données. Ces difficultés ont contribué à renforcer nos compétences en analyse, en résolution de problèmes et en développement logiciel.

Ce projet nous a également permis d'acquérir une expérience concrète du travail collaboratif et de l'utilisation d'outils professionnels tels que Git et GitHub pour la gestion des versions. Enfin, plusieurs améliorations pourraient être envisagées dans le futur, telles que l'intégration d'une authentification plus sécurisée, la génération automatique de bulletins PDF, l'ajout d'un système de notifications ou encore le développement d'une version mobile de l'application.

En conclusion, la réalisation de UniSchool constitue une expérience particulièrement enrichissante qui nous a permis de consolider nos compétences techniques et de mieux comprendre les différentes étapes de conception et de développement d'une application de gestion complète.

Références

Références et ressources

Documentation

- JavaFX Documentation : <https://openjfx.io/>
- Oracle Java Documentation : <https://docs.oracle.com/en/java/>
- MySQL Documentation : <https://dev.mysql.com/doc/>
- MySQL Connector/J : <https://dev.mysql.com/downloads/connector/j/>
- IntelliJ IDEA Documentation : <https://www.jetbrains.com/idea/documentation/>

Tutoriels et ressources

- JavaFX Tutorial : <https://www.javatpoint.com/javafx-tutorial>
- JDBC Tutorial : <https://www.javatpoint.com/jdbc-tutorial>
- MVC Architecture in Java : https://www.tutorialspoint.com/design_pattern/mvc_pattern.htm
- Scene Builder Documentation : <https://gluonhq.com/products/scene-builder/>

Outils utilisés

- IntelliJ IDEA : environnement de développement intégré (IDE).
- JavaFX : développement des interfaces graphiques.
- MySQL : système de gestion de base de données.
- XAMPP : serveur local intégrant Apache, MySQL et phpMyAdmin.
- phpMyAdmin : administration et gestion de la base de données.
- Git et GitHub : gestion des versions et collaboration.
- Scene Builder : conception visuelle des interfaces FXML.

Liens du projet

- Vidéo de démonstration : <https://drive.google.com/drive/folders/1OKnxs5jlokKc4JpYgklrkyAFwi-FpxOK?usp=sharing>